

חלק מס' 1 יסודות פייתון למפתחים

Data types & Variables

1. Variables (משתנים)

תווית (Label) או "שלט הכוונה" שמצביעה על אובייקט שנמצא אי שם בזיכרון של המחשב. כאשר מקצים למשתנה את הערך בעזרת סימן השוויון (=), פייתון מזהה את סוג הנתון שהמשתנה יחזיק.

```
browser_name = "Chrome"
max_retries = 3
is_test_passed = True
```

דוגמאות:

פקודת print()

המילה print באנגלית משמעותה "הדפס".

כשאנחנו כותבים print ושמים את שם המשתנה (התווית) בתוך הסוגריים (), אנחנו בעצם אומרים למחשב: "תציג לי על המסך את האובייקט שהשלט הזה מצביע עליו".

```
browser_name = "Chrome"
print(browser_name)
```

דוגמה 1:

הפקודה print את מה שבסוגריים browser_name תהיה "Chrome".

```
max_retries = 3
print(max_retries)
```

דוגמה 2:

הפקודה print את מה שבסוגריים max_retries תהיה 3.

```
is_test_passed = True
print(is_test_passed)
```

דוגמה 3:

הפקודה print את מה שבסוגריים is_test_passed תהיה True.

בתוך הסוגריים של ה-print לא שמנו מירכאות מסביב לשם המשתנה.

פקודת הדפסת משתנה עם מירכאות, סימן לטקסט, תדפיס את המשתנה כטקסט.

פקודת הדפסת משתנה ללא מירכאות, סימן לבדיקת ערך המשתנה, תדפיס את ערך המשתנה.

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python datatypes_practice.py
• Chrome
3
True
```

חלק מס' 1 יסודות פייתון למפתחים

Data types & Variables

2. Basic Data Types (סוגי נתונים בסיסיים)

אלו ארבעת אבני הבניין לכל טסט שכותבים:

String (str)

מחרוזת טקסט.

כל מה שמוקף בגרשיים בודדים (') או כפולים (").

באוטומציה, כל טקסט ששולפים מאתר אינטרנט (כמו כותרת, או מחיר) יחזור תמיד כ-String.

Integer (int)

מספר שלם, חיובי או שלילי, ללא נקודה עשרונית.

למשל 5, -100.

Float (float)

מספר עשרוני.

למשל 3.14, 19.99.

Boolean (bool)

ערך בוליאני המייצג אמת או שקר.

קריטי לתנאים וליצירת Assertions (אימותים בטסטים).

הערכים האפשריים היחידים הם True או False (חובה אות גדולה בהתחלה, אחרת פייתון יזרוק שגיאה).

דוגמאות

1 String (str) - מחרוזת טקסט (במרכאות)

```
page_title = "Welcome to our store"
error_message = 'Invalid password'
```

2 Integer (int) - מספר שלם (בלי נקודה)

```
status_code = 200
items_in_cart = 5
```

3 Float (float) - מספר עשרוני (עם נקודה)

```
product_price = 49.99
load_time_seconds = 1.5
```

4 Boolean (bool) - אמת או שקר (חובה אות ראשונה גדולה!)

```
is_button_enabled = True
is_error_displayed = False
```

חלק מס' 1 יסודות פייתון למפתחים

Data types & Variables

דוגמה type()

כדי לראות את סוג הנתון על המסך, משלבים את פקודת type() בתוך פקודת ההדפסה print()

```
is_button_enabled = True
is_error_displayed = False

page_title = "Welcome to our store"
print(type(page_title))

status_code = 200
print(type(status_code))

product_price = 49.99
print(type(product_price))

is_button_enabled = True
print(type(is_button_enabled))
```

הפקודה בודקת איזה סוג נתון מסתתר מאחורי התווית product_price ואז מדפיסה את התשובה למסך.

התוצאה

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python datatypes_practice.py
• <class 'str'>
  <class 'int'>
  <class 'float'>
  <class 'bool'>
```

כלומר, זוהי מספר עשרוני- float

בדיקת סוג נתון בטסטים אמיתיים

רק כאשר שואבים נתון מבחון ולא שולטים עליו.

למשל, כשהטסט מושך מחיר של מוצר מהאתר, ורוצים לוודא שפייתון זיהה אותו כמספר עשרוני (float) ולא בטעות כטקסט (str).

אם הוא שמור כטקסט, לא נוכל לבצע עליו פעולות חשבון בטסט (כמו לבדוק אם המחיר גדול מ-0).

חלק מס' 1 יסודות פייתון למפתחים

Data types & Variables

3. Casting (המרת טיפוסים)

חוק ברזל באוטומציה :

אין לנו שליטה על שום נתון שאנחנו שולפים מאתר אינטרנט (מה- UI של הדפדפן), הכל מגיע אלינו לקוד תמיד כטקסט (String), גם אם בעין הוא נראה לנו כמו מספר, כך דפדפנים עובדים.

לכן, כדי שנוכל לבצע פעולות מתמטיות על מחירים, תאריכים או כמויות ששאבנו מהאתר, חייבים להמיר אותם אצלנו בקוד מ"טקסט חסר משמעות" למספרים אמיתיים.

כאן האתגר האמיתי בעבודה מול UI .

נניח שהטסט שולף מחיר של מוצר מהאתר, פייתון רואה את זה כ-"50" (String) .

אם מחברים לזה את המספר 10 (Integer) כדי לבדוק עלות משלוח, פייתון יקרוס .

אי אפשר לעשות פעולות מתמטיות בין טקסט למספר.

הפתרון Casting :

המרה יזומה של הנתון לסוג אחר.

int ("50") יהפוך את המחרוזת למספר שלם.

str (100) יהפוך את המספר למחרוזת טקסט.

float ("19.9") יהפוך מחרוזת למספר עשרוני.

דוגמאות

1. המחיר ששלפנו מהאתר (פייתון מתייחסת אליו כטקסט בגלל המירכאות)

```
item_price_from_ui = "50"
```

2. עלות המשלוח שלנו (מספר שלם אמיתי)

```
shipping_cost = 10
```

3. שלב ההמרה (Casting): לוקחים את הטקסט, הופכים למספר, ושומרים במשתנה חדש

```
real_price = int(item_price_from_ui)
```

4. עכשיו אפשר לחשב את הסכום הכולל בטסט בלי שהקוד יקרוס!

```
total_cost = real_price + shipping_cost
```

התוצאה שתודפס על המסך תהיה 60

חלק מס' 1 יסודות פייתון למפתחים

Data types & Variables

```
print(total_cost)
```

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python datatypes_practice.py  
60
```

הסבר

במקום לנסות לחבר טקסט ומספר (מה שהיה נותן שגיאה), עוטפים את המשתנה של הטקסט בפקודה `int()`, פייתון מחלץ משם את המספר 50, מאפשר חישוב מתמטי תקין.

שליפת מחיר עם אגורות, למשל "19.99", עובד לגמרי זהה, רק עם `float`

דוגמא

```
real_price = int(item_price_from_ui)
```

במקום

```
real_price = float(item_price_from_ui)
```

 נשתמש ב-

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python datatypes_practice.py  
29.99
```

חוק ברזל באוטומציה

- נתונים שלמים (למשל: כמות מוצרים בעגלה, מספר טיסות שנמצאו) ממירים עם `int()`
- כסף ומחירים (נתונים שיש בהם נקודה עשרונית) ממירים **תמיד** עם `float()`

חלק מס' 1 יסודות פייתון למפתחים

Data types & Variables

4. Mutability (השתנות) - חובת הבנה!

נושא שמפיל מועמדים בראיונות עבודה .

בפייתון, כל אובייקט מתחלק לשני סוגים

Mutable (ניתן לשינוי) ו- Immutable (בלתי ניתן לשינוי) .

ארבעת סוגי הנתונים שלמדנו הרגע (String, Integer, Float, Boolean) הם כולם ! Immutable

זה אומר שאם יצרנו משתנה טקסט, הזיכרון שבו שמור הטקסט הזה נעול .

אם ננסה "לשנות" אותו בקוד, פייתון לא משנה את האובייקט המקורי.

במקום זאת, פייתון יוצרת אובייקט חדש לגמרי בזיכרון, ומעבירה את התווית (המשתנה) להצביע עליו.

האובייקט הישן יימחק על ידי מנגנון איסוף הזבל (Garbage Collector) .

חוסר הבנה של העיקרון הזה יגרום לכתיבת קוד לא יעיל ואיטי.

איך Immutable פוגש אותנו באוטומציה

כיוון שמחרוזת טקסט (String) היא Immutable, אי אפשר לשנות אותה "במקום".

אם נשלוף מהאתר מחיר עם דולר ("50\$") ונפעיל עליו פקודת ניקוי (כמו replace) שמסירה את הדולר - פייתון לא תשנה את המשתנה המקורי! היא תיצור בזיכרון אובייקט חדש ונקי ("50").

אם לא "נתפוס" את התוצאה החדשה ונשמור אותה במפורש לתוך משתנה (נדרוס את התווית הקיימת או ניצור חדש), הטסט שלנו ימשיך להחזיק את המחרוזת הישנה עם הדולר, והקוד יקרוס כשננסה לעשות לה Casting למספר כדי לבדוק אותה.

דוגמא

```
price_from_ui = "$50"
```

--- הטעות הנפוצה (קוד שיקרוס) ---

נותנים פקודה להחליף את סימן הדולר בכלום (למחוק אותו)

```
price_from_ui.replace("$", "")
```

חלק מס' 1 יסודות פייתון למפתחים

Data types & Variables

אם נדפיס עכשיו, נגלה ששום דבר לא השתנה! כי סטרינג הוא Immutable.
פייתון יצרה "50" נקי בזיכרון, אבל אף משתנה לא תפס אותו, אז הוא נמחק (Garbage Collector).

ידפיס: \$50 (הטסט בדרך להתרסק)
`print(price_from_ui)`

--- הדרך הנכונה (לתפוס את האובייקט החדש) ---
לוקחים את האובייקט החדש שנוצר ודורסים איתו את התווית הקיימת:

`price_from_ui = price_from_ui.replace("$", "")`
ידפיס: 50 (עכשיו אפשר לעשות לזה (int()
`print(price_from_ui)`

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python datatypes_practice.py
$50
○ 50
```

השורה הראשונה (\$50)

ההוכחה לטעות הנפוצה ולזה שטקסט הוא **Immutable**, פקודה להעיף את הדולר, אבל בגלל אי שמירת התוצאה, המשתנה לא השתנה, ופייתון הדפיסה את המחרוזת המקורית כאילו לא קרה כלום.

השורה השנייה (50)

ביצענו עדכון של המשתנה.

כעת, המשתנה `price_from_ui` מצביע על מחרוזת נקייה מתווים מיוחדים.

רק במצב כזה, ניתן להמיר את המחרוזת בהצלחה למספר שלם באמצעות פקודת ההמרה `int()` מבלי שפייתון תחזיר שגיאה.

---#תרגיל מסכם: טסט עגלת קניות---

1# יצירת משתנה (Variables & Data Types)

צרי משתנה בשם `product_name` שיחזיק את המחרוזת "Laptop" (String)

חלק מס' 1 יסודות פייתון למפתחים

Data types & Variables

2# צרי משתנה בשם is_in_stock שיחזיק את הערך הבוליאני של אמת (True) זכרי אות

ראשונה גדולה!

3# המרת טיפוסים (Casting) של מספר שלם

הנה נתון (כמות) ששלפנו מה- UI של האתר והגיע כטקסט:

```
ui_quantity = "3"
```

צרי משתנה חדש בשם real_quantity, ותבצעי עליו המרה (Casting) של ui_quantity

למספר שלם.

4# השתנות (Mutability)

הנה מחיר ששלפנו מה- UI והגיע כטקסט עם סימן של דולר:

```
ui_price = "$120.50"
```

בגלל שסטרינג הוא Immutable, השתמשי בפקודת הניקוי ("\$", "") replace על

המשתנה ui_price,

ותדאגי *לדרוס* את התווית המקורית כדי לשמור את האובייקט הנקי שנוצר!

5# המרה למספר עשרוני (Casting)

צרי משתנה חדש בשם final_price, ותבצעי בו המרה (Casting) של ui_price הנקי

למספר עשרוני.

6# בדיקת סוג נתון (type) והדפסה (print)

חלק מס' 1 יסודות פייתון למפתחים

Data types & Variables

כדי להוכיח שההמרות הצליחו, הדפיסי למסך את סוג הנתון (type) של המשתנה

real_quantity

#7 הדפיסי למסך את סוג הנתון (type) של המשתנה final_price

```
product_name = "Laptop"
is_in_stock = True
ui_quantity = "3"
real_quantity = int(ui_quantity)
ui_price = "$120.50"
ui_price = ui_price.replace("$", "")
final_price = float(ui_price)
print(type(real_quantity))
print(type(final_price))
```

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python datatypes_practice.py
<class 'int'>
<class 'float'>
```

--- תרגיל רמה 2: טסט למערכת הזמנת טיסות ---

חלק 1: יצירת משתנים

1. צרי משתנה בשם is_direct_flight ושימי בו ערך בוליאני שאומר "שקר" (שימי לב לאותיות!).

חלק 2: חשיבה עצמאית על Casting

שלפנו מהאתר כמות של כרטיסי טיסה שהלקוח בחר:

ui_tickets = "5"

2. צרי משתנה בשם real_tickets, ותעשי המרה (Casting) ל-ui_tickets.

האתגר: תחליטי לבד לאיזה סוג נתון ממירים כמות של אנשים (int או float?), ותכתבי את הפקודה הנכונה.

חלק מס' 1 יסודות פייתון למפתחים

Data types & Variables

חלק 3: אתגר הדיבאג (איתור שגיאות) - Mutability

מפתח ג'וניור בצוות ניסה לנקות את המחיר ולהמיר אותו, אבל הקוד שלו יקרוס כשירוץ!

הנה הקוד השגוי שלו:

```
#ui_price = "$499.99"
```

```
#ui_price.replace("", "$")
```

```
#final_price = float(ui_price)
```

3. הראי לג'וניור איך עושים את זה נכון! כתבי מחדש את שלושת השורות האלו

כך שהדולר באמת ינוקה מהמשתנה (תזכרי מה קורה כשהאובייקט הוא Immutable) והערמות יעבדו.

חלק 4: הוכחה

4. הדפיסי למסך את סוג הנתון (type) של המשתנה real_tickets

5. הדפיסי למסך את סוג הנתון (type) של המשתנה final_price

```
is_direct_flight = False
ui_tickets = "5"
real_tickets = int(ui_tickets)
ui_price = "$499.99"
ui_price = ui_price.replace("$", "")
final_price = float(ui_price)
print(type(real_tickets))
print(type(final_price))
```

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python datatypes_practice.py
<class 'int'>
<class 'float'>
```

--- #תרגיל רמה 3 (Boss Level): טסט לפיצ'ר "סיכום תיק חכם" ---

#חלק 1: סוגי נתונים והמרה מתוך ה-UI

#המערכת סרקה את התיק הרפואי והחזירה את הגיל של המטופלת כטקסט:

```
ui_patient_age = "42"
```

חלק מס' 1 יסודות פייתון למפתחים

Data types & Variables

1. #צרי משתנה בשם real_age ותמירי אליו את הגיל בצורה נכונה .
#(עלייך להחליט לבד איזה סוג נתון מתאים לגיל, ולכתוב את ההמרה המתאימה.)
#כתבי את הקוד שלך כאן מתחת:

```
real_age = int(ui_patient_age)
```

2. #צרי משתנה בשם is_summary_ready ושימי בו ערך בוליאני שאומר
"אמת."
#כתבי את הקוד שלך כאן מתחת:

```
is_summary_ready = True
```

#חלק 2: המלכודת של ראיונות העבודה (Mutability)
#שלפנו מהמערכת את עלות הטיפול הכוללת.
#היא מגיעה כטקסט עם סמל של שקלים:

```
ui_total_cost = "₪1250.50"
```

#הפעם, במקום לדרוס את התווית, מתכנת אחר בצוות כתב קוד שמנקה את הסמל
#ושומר את התוצאה בתוך משתנה *חדש* לגמרי:

```
clean_cost = ui_total_cost.replace("₪", "")
```

#ומיד לאחר מכן ביצע המרה תקינה כדי שנוכל לבדוק את המחיר:

```
final_cost = float(clean_cost)
```

3. #אתגר חשיבה (לענות מהראש בלבד, בלי קורסור!):
#המראיין מסתכל עלייך ושואל: "אם נכתוב עכשיו את הפקודה
print(ui_total_cost), מה בדיוק יודפס על המסך?"

חלק מס' 1 יסודות פייתון למפתחים

Data types & Variables

#תכתבי לי כאן למטה בהערה (עם) #מה יודפס, ותסבירי במשפט אחד *למה* זה מה שיודפס

#תוך שימוש בחומר שלמדנו על Immutable.

#התשובה שלך:

#חלק 3: ייעול קוד והבנת הזיכרון (DRY)

#בסוף הטסט, אנחנו צריכים להדפיס למסך את סוג הנתון (type) של הגיל כדי לוודא שלא איבדנו אותו.

#מפתח מתחיל כתב את שתי השורות האלו כדי להדפיס את זה:

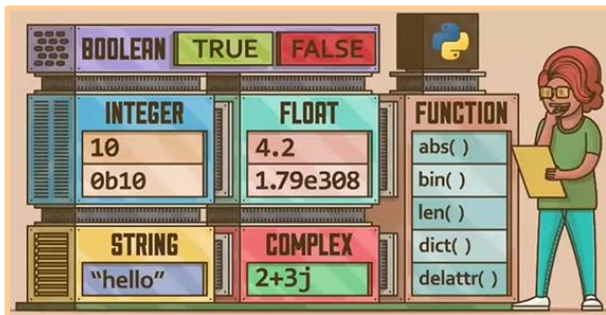
```
# age_for_print = int(ui_patient_age)
```

```
# print(type(age_for_print))
```

4. #בהסתמך על מה שכבר עשית בשאלה 1 למעלה, איך היית כותבת את זה עכשיו

#בשורת קוד אחת בלבד, נקייה וכלי כפילויות של המרות?

#כתבי את הקוד המתוקן שלך כאן מתחת:



Data Types

אחידות

לכל שפת תכנות יש **Data Types**, ב-Python יש :

String (str) – טקסט, למשל: "Hello", "password123".

Int – מספר שלם, למשל: -3, 42.

Float – מספר עשרוני, למשל: 0.5, 19.99.

List – מאחסן אוסף של ערכים, למשל: ["Dave", "Maria", "James"]

Tuple – מאחסן אוסף של ערכים נעולים, למשל: ("apple", "banana", "cherry")

Dictionary – מאחסן אוסף של Key:values, למשל: {"username": "admin", "password": "1234"}

Boolean – מייצג ערכים שהם True או False, למשל: is_logged_in = True

Is_admin = False

Data Types

&

Variables

אחיהוד

Variables

בעברית נקראים : "משתנים".

אנחנו קובעים את השם שלהם .

יאחסנו בתוכם את ה-Data Type שנקבע.

לדוגמא:

```
password = "Aa123"  
print(password)
```

Password זה השם של המשתנה (Variable)

הערך של Password הוא: "Aa123"

ה-Data type של Password זה String .

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python datatypes.py  
Aa123
```

כלל ברזל :

נקרא בפיתוח Top Down Execution (שורה אחרי שורה).

הקוד בפיתוח מפורש ורץ שורה אחרי שורה, מלמעלה למטה .

כלומר, חובה קודם להגדיר משתנה, ורק אז אפשר להשתמש בו.

למשל : אם בשורה 1 נגדיר את המשתנה password = "Aa123", ובשורה 2 ניתן פקודת הדפסה - print(password) הקוד יעבוד חלק.

אבל, אם נהפוך את הסדר, נבקש להדפיס בשורה 1 ונגדיר את המשתנה רק בשורה 2 פייתון יקרוס ויזרוק לנו שגיאה מסוג NameError, הסיבה לכך היא שבזמן שפייתון קרא את פקודת ההדפסה בשורה 1, המשתנה password עדיין לא היה קיים בזיכרון.

Dynamic Typing

אחיהוד

Dynamic Typing

Python משתמשת ב-Dynamic Typing (לא היחידה) להבדיל משפות תכנות אחרות כמו Java, ב-Python אנחנו לא צריכים "להודיע" באיזה Data Type אנחנו משתמשים. השימוש ב-Semicolon ";" גם מיותר ב-Python, ירידת שורה מספיקה.

למשל ב-Java :

```
Int age = 25;  
Float price = 19.99;  
String name = "alice";
```

ב-Python :

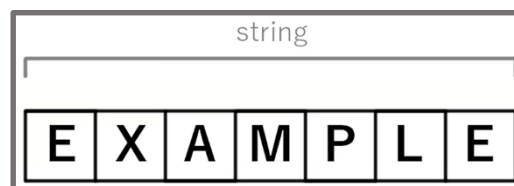
```
Age = 25  
Price = 19.99  
Name = "Alice"
```

Exercise-String

אחיהוד

תרגיל String

1. הגדירו משתנה שהערך שלו זה השם שלכם והדפיסו אותו.
2. הגדירו משתנה שערכו מספר ידוע שאסור לעשות עליו פעולות חשבוניות, והדפיסו אותו.
3. הגדירו שני משתנים : שם משפחה, עיר מגורים – והדפיסו אותם.



```
my_name_is = "Anna"  
print(my_name_is)  
  
my_phone_number_is = "035528323"  
print(my_phone_number_is)  
  
my_last_name_is = "Rubenchik"  
city = "Tel Aviv"  
print(my_last_name_is, city)
```


Exercise-Integer

אחיהוד

תרגיל Int

1. הגדירו משתנה Int "my_age" והדפיסו אותו .
2. הגדירו שני משתני Int, בצעו עליהם פעולת חיבור והדפיסו את התוצאה.
3. הגדירו שלושה משתני Int, בצעו עליהם פעולת כפל והדפיסו את התוצאה.
4. הגדירו משתנה ששווה ל-20 ועוד משתנה ששווה ל-3 ובצעו פעולת חילוק, מה התוצאה שקיבלתם ?

Operator	Operation
+ (Addition)	$z = x + y$
- (Subtraction)	$z = x - y$
* (Multiplication)	$z = x * y$
/ (Division)	$z = x / y$

```
my_age = 35
print(my_age)

my_age = 35
current_year_is = 2026
print(my_age+current_year_is)

my_age = 35
current_year_is = 2026
street_number_is = 89
print(my_age*current_year_is*street_number_is)

number_of_books_that_i_have_read = 20
number_of_books_that_i_have_not_like = 3
print(number_of_books_that_i_have_read/number_of_books_that_i_have_not_like)
```

Exercise-Float

אחיהוד

תרגיל Float

1. הגדירו משתנה שנקרא "age" עם הגיל הנוכחי שלכם, כולל מספר החודשים, והדפיסו אותו.
2. הגדירו מספר במשתנה a, מספר במשתנה b, והתוצאה שלהם במשתנה c, בצעו פעולה חשבונית כלשהי, כך שהתוצאה של c תצא 10.4.

Float

10.123

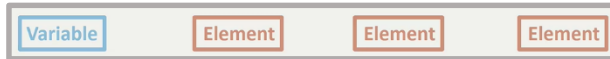
```
my_age = 35.5  
print(my_age)
```

```
a = 5  
b = 5.4  
c = (a+b)  
print(c)
```

Data Type - List

אחידוד

מה זה List



משתנה שבתוכו יש אוסף של ערכים .

יכול לאחסן בתוכו כל Data Type (אפילו יכול לאחסן עוד list בתוכו).

```
names = ["Dave", "Maria", "James"]
numbers = [1, 3, 5, 7]
user_info = ["Daniel", 30, ["Rotem", "Shani"]]
```

כותבים בסוגריים [מרובעים].

Syntax : מפרידים ערכים בפסיק.

Exercise - List

תרגיל List

1. הגדירו list שנקרא "testingLevels" ובתוכו הכניסו 3 רמות בדיקה , הדפיסו

את הרמת בדיקה הכי מוקדמת ב-SDLC .

2. הגדירו list שמכיל בתוכו 3 מספרים של status code :

מספר ראשון : סטטוס קוד שמראה שגיאה בצד הלקוח .

מספר שני : סטטוס קוד שמראה שמידע חדש נוצר בהצלחה .

מספר שלישי : סטטוס קוד שמראה שגיאה בצד השרת .

הדפיסו רק את הסטטוס קוד השלישי.

3. הגדירו 3 lists שמכילים כולם לפחות 3 ערכים של פירות/ירקות/ערים

(לבחירתכם).

הגדירו list רביעי שיכיל בתוכו את שלושת ה-lists שהכנתם לפניו.

הדפיסו רק את הערך האחרון ב-list השני.

```
print(your_answer[5])
```

פתרון

פתרון השגיאה IndexError: list index out of range

זו כנראה השגיאה הכי נפוצה בעולם התכנות.

הפירוש שלה בעברית פשוטה ניסינו לשלוף פריט ממיקום (אינדקס) שלא קיים ברשימה .

זה כמו לבקש את הספר השישי ממדף שיש עליו רק 5 ספרים.

נספור יחד את האינדקסים (המיקומים) ברשימת ה-user_list .

בפייתון סופרים מאפס:

אינדקס [0] = "Apple"

אינדקס [1] = "Banana"

אינדקס [2] = "Coconut"

אינדקס [3] = ["Tomato", "Cucumber", "Onion"] (כל רשימת הירקות נחשבת לפריט אחד)

אינדקס [4] = ["Tel_Aviv", "Bat_Yam", "Kfar_Sabba"] (כל רשימת הערים נחשבת לפריט אחד)

Data Type - List

אחידות

הגענו לאינדקס 4, והרשימה נגמרה.

כשאמרנו לפייתון `print(user_list[5])`, הוא חיפש את הפריט השישי, לא מצא כלום, וקרא עם שגיאת חריגה מגבולות הרשימה (Out of range).

: Code Review

סעיף 1 (רמות בדיקה)

מה עשינו `testingLevels = [1, 2, 3]`

מבחינת קוד פייתון - הפתרון עובד פרפקט ואינדקס 0 הודפס.

אבל רצינו את רמות הבדיקה ב- SDLC (מחזור חיי פיתוח תוכנה) הן מונחים מקצועיים (טקסט), לא מספרים.

הרמה הכי מוקדמת היא בדיקות יחידה (Unit Testing), איך נכון לכתוב:

```
testingLevels = ["Unit Testing", "Integration Testing", "System Testing"]
print(testingLevels[0])
```

סעיף 2 (סטטוס קוד)

מה עשינו `status_code = [201, 400, 500]`

שלפנו נכון את אינדקס 2! אבל חשוב לשים לב לדרישות:

מספר ראשון: שגיאת לקוח (למשל 400 או 404).

מספר שני: יצירה בהצלחה (201).

מספר שלישי: שגיאת שרת (500).

שמנו את 201 כראשון.

באוטומציה קוראים את הדרישות ונצמדים אליהם כמו פלס.

```
status_codes = [400, 201, 500]
print(status_codes[2])
```

כתיבה נכונה :

סעיף 3 הרשימה המורכבת

נדרש ליצור רשימה רביעית שתכיל בתוכה את המשתנים של שלושת הרשימות הקודמות. במקום לכתוב את הפירות והירקות מחדש, אנחנו פשוט משתמשים בשמות של המשתנים.

בניית רשימה בתוך רשימה (Nested List)

אנחנו צריכים לתת לפייתון הוראות ניווט בשני שלבים.

Data Type - List

אחידות

אנחנו צריכים להגיד לו קודם תפתח את המגירה הנכונה, ואז תמשוך משם את הפריט הנכון.
פיתון קורא את הסוגריים משמאל לימין, כמו רובוט (`print(all_lists[1][2])`).

שלב ראשון - מתייחסים לסוגריים השמאליים [1]

פיתון שואל לאיזו מגירה בארון ללכת?

אומרים לו למגירה [1], שזו רשימת הירקות, הרשימה השנייה.

עכשיו פיתון מחזיק ביד את המגירה הזאת ["Tomato", "Cucumber", "Onion"]

שלב שני - מתייחסים לסוגריים הימניים [2]

פיתון שואל אוקיי, פתחתי את המגירה, איזה פריט בפנים את רוצה?

אומרים לו, את הפריט באינדקס [2], בצל - הערך האחרון, כי סופרים 0, 1, 2.

פיתון שולף את הבצל ומדפיס אותו!

סיכום

כשיש רשימה בתוך רשימה, מספר אחד בסוגריים מרובעים לא מספיק.

הסוגריים הראשונים פותחים את הרשימה הגדולה (בוחרים באיזו רשימה משלוש הרשימות אנחנו רוצים לגעת).

הסוגריים השניים בוחרים את הפריט הספציפי מתוך הרשימה שכבר בחרנו.

כלומר, אינדקס כפול (דו ממדי):

קודם נכנסים לרשימה השנייה (הירקות): `all_lists[1]`

ובתוכה, שולפים את הפריט האחרון (בצל, אינדקס 2): `[2]`

```
fruits = ["Apple", "Banana", "Coconut"]
vegetables = ["Tomato", "Cucumber", "Onion"]
cities = ["Tel_Aviv", "Bat_Yam", "Kfar_Sabba"]

# יוצרים רשימה שמכילה את המשתנים עצמם
all_lists = [fruits, vegetables, cities]
print(all_lists[1][2])
```

מחברים אותם ביחד:

```
testingLevels = [1, 2, 3]
print(testingLevels[0])

testingLevels = ["Unit Testing", "Integration Testing", "System Testing"]
print(testingLevels[0])

status_code = [201, 400, 500]
print(status_code[2])

status_codes = [400, 201, 500]
print(status_codes[2])

fruits = ["Apple", "Banana", "Coconut"]
vegetables = ["Tomato", "Cucumber", "Onion"]
cities = ["Tel_Aviv", "Bat_Yam", "Kfar_Sabba"]
```

Indexing – List

אחיהוד

איך סופרים בכתיבת קוד

בשפות פיתוח, נהוג לספור מהספרה "0" ולא מהספרה "1" כמו שאנחנו רגילים בחיי היומיום שלנו.

כדי להתייחס לאלמנט ספציפי בתוך list או data type אחר שיועד להכיל כמה ערכים (dict), נשתמש ב-indexing.

```
index = 0 1 2 3 4
numbers = [1, 2, 3, 4, 5]
print(numbers[1])
```

Data Type – Tuple

אחיזה

מה זה Tuple

משתנה שבתוכו יש אוסף של ערכים שלא ניתן לשנות שלא מהמקור.
יכול לאחסן בתוכו ערכים מכל Data Type (אפילו עוד Tuple).
כותבים בסוגריים (עגולים).

גם לו יש indexing

Syntax : מפרידים ערכים בפסיק.

```
my_tuple = ("apple", "banana", "cherry")  
my_tuple[0] = "strawberry" <-- לא יעבוד
```

הערכים ב-tuple הם read only .

יהיה אפשר לשנותם רק מאיפה שהגדרנו לראשונה.

ההבדל בין List ל-Tuple זהה להבדל בין קובץ Word לקובץ PDF

List רשימה עם סוגריים מרובעים []

זה כמו קובץ Word .

יוצרים אותו, ומאוחר יותר בקוד ניתן להחליט שמוחקים פריטים, מוסיפים פריטים חדשים, או משנים לפריט קיים את השם, זה גמיש ומשתנה.

Tuple עם סוגריים עגולים ()

זה כמו קובץ PDF חתום ונעול .

ברגע שיוצרים ושומרים – זהו! ננעל !

הקוד לא יכול להוסיף לו כלום, לא יכול למחוק ממנו כלום, ולא יכול לשנות שום פריט בתוכו בזמן שהתוכנה רצה.

פירוש המילה

השם "Tuple" מגיע ממתמטיקה.

כמו שיש זוג (Couple) או שלישייה (Triple), מתמטיקאים היו צריכים מילה לתיאור של כמה פריטים שמחוברים יחד בסדר מסוים, לכן הם המציאו את המילה Tuple .

בפיתון זה פשוט אומר רשימה נעולה.

למה שנרצה רשימה שאי אפשר לשנות?!?

זו השאלה הכי חשובה! הרי אם List זה גמיש, למה להגביל את עצמנו ?

התשובה היא :הגנה מפני באגים של עצמנו .

Data Type – Tuple

אחיזה

מפתחים לפעמים מגדירים נתונים שממש, אבל ממש לא רוצים ששום פקודת קוד בהמשך תשנה אותם בטעות (כי אם הם ישתנו, כל המערכת תקרוס).

ה"נעילה" של ה-Tuple מבטיחה שזה יהיה Read Only.

באילו מקרים משתמשים בזה

3 דוגמאות קלאסיות מחיי היום יום של פיתוח אוטומציה:

נקודות ציון על המסך (קואורדינטות)

נניח שכותבים אוטומציה שמזיזה את העכבר ולוחצת על כפתור "שלח" במיקום מדויק במסך :
(x=150, y=300), הנקודה הזו לעולם לא משתנה.

שומרים אותה ב-Tuple.

אם בטעות ננסה לכתוב קוד שמשנה את ה-x ל-200, פייתון יזרוק שגיאה ויעצור אותנו מלהרוס את הטסט.

פרטי התחברות לשרת בדיקות (DB)

כדי להתחבר ל-DB דרך פייתון, צריכים כתובת IP ופורט.

הנתונים האלה קבועים (8080, "192.168.1.1").

שמים אותם ב-Tuple כדי ששום קוד אחר לא ידרוס אותם בטעות.

רשימת חודשי השנה או ימי השבוע

לשבוע תמיד יש 7 ימים, והם תמיד באותו סדר.

אין שום סיבה בעולם שנוסיף יום חדש לרשימה, לכן ימי שבוע ישמרו תמיד כ-Tuple

איך משתמשים (Indexing)

זהה לחלוטין ל-List.

במידה ויש Tuple של ימים ("Sunday", "Monday", "Tuesday")

ורוצים להדפיס את היום הראשון, כותבים את שם המשתנה ולידו סוגריים מרובעים עם המיקום
`print(days[0])`

ולכן הכללים של השליפה הם אותם כללים,

רק הבנייה היא עם סוגריים עגולים

והרשימה היא "לקריאה בלבד".

`variable = ( ("", ""), ("", "") )`

Exercise – Tuple

אחיהוד

תרגיל Tuple

1. הגדירו Tuple שמכיל בתוכו ערכי String של כל הכותרות בתיעוד באג שאנחנו ממלאים.

לדוגמא : title ,description, etc.... .

הדפיסו רק את severity .

2. הגדירו Tuple שנקרא "box testing" ובתוכו, הכניסו שני Tuples :

בראשון יהיה שתי בדיקות של קופסה שחורה.

בשני יהיה שתי בדיקות של קופסה לבנה.

הדפיסו רק את Unit Test .

```
my_tuple = ("apple", "banana", "cherry")
```

```
my_tuple = ("title", "discription", "steps_to_reproduce", "actual_result", "severity")
print(my_tuple[4])

(טופל קופסה שחורה) 1 מגירה, (טופל קופסה לבנה) 2 מגירה
box_testing = ( ("Sanity", "System"), ("Code_Coverage", "Unit Test") )
print(box_testing[1][1])
```

Data Type – Dictionary

אחיהוד

מה זה Dict

- מאחסן זוגות של Key:Value (כמו JSON) .
- כותבים בסוגריים {מסולסלים} (בשונה מ-List שכותבים בסוגריים מרובעים).
- משתמשים בנקודתיים ובפסיק ב-Syntax .
- ה-Key חייב להיות Unique .
- אי אפשר לכתוב את אותו שם של Key יותר מפעם אחת.
- ה-Key יכול להיות String ו-Int .
- ה-Key לא יכול להיות List או Dictionary .
- ה-Value יכול להיות כל סוג של Data Type .
- אי אפשר להשתמש ב-Indexing ב-Dict .

```
user_information = {"first_name": "John",  
                   "last_name": "Smith",  
                   "age": 54,  
                   "id": "5432553"  
}
```

Exercise – Dictionary

אחיהוד

תרגיל Dict

1. הגדירו משתנה בשם "Mobile_std" (תנו את שם הפרויקט מובייל שלכם)

Key אחד בשם "Feature" (תנו שם של פיצ'ר שבדקתם)
Key שני בשם "Bugs" (ציינו כמה באגים היו לכם)
Key שלישי בשם "testing_type" (ציינו סוג בדיקה שהשתמשתם)
הדפיסו את המשתנה במלואו.

2. הגדירו משתנה בשם "my_team"
Key אחד בשם "team_member1" (תנו שם של בעל תפקיד בצוות הייטק שלנו)
צרו לפחות 3 Keys עם שמות של בעלי תפקידים.
הדפיסו את בעל התפקיד שהכי קשור ל-SRS (Software Requirements Specification) מסמך אפיון.

```
user_information = {"first_name": "John",  
                    "last_name": "Smith",  
                    "age": 54,  
                    "id": "5432553"  
}  
  
print(my_team ["team_member1"])
```

```
Rami_Levi_Mobile_std = {"feature": "WhatsApp_Share",  
                        "bugs": 16,  
                        "testing_type": "regression",  
                        }  
  
print(Rami_Levi_Mobile_std)  
  
my_team = {"team_member1": "team_leader",  
           "team_member2": "Scrum_head",  
           "team_member3": "manager",  
           }  
  
print(my_team ["team_member2"])
```

Data Type – Boolean



אחיהוד



מה זה בעצם Boolean

סוג נתונים (Data Type) שמייצג תשובה החלטית של "כן" או "לא".
מחשבים לא מבינים "אולי", "כמעט", או "לפעמים".
הם מבינים רק מצבים בינאריים מוחלטים (אפס או אחד).
Boolean הוא הכלי שלנו לשאול את הקוד שאלות לוגיות, ולקבל עליהן תשובה חותכת וסופית.

תפקידו באוטומציה

אפשר להשוות אותו לגלאי (סנסור) של הטסטים שלנו.
הקוד שנכתוב כל הזמן ישאל את המערכת שאלות וישווה נתונים:
"האם כפתור ההתחברות מופיע כרגע על המסך?"
"האם ההודעה שקפצה שווה לטקסט שציפינו לקבל?"
"האם מספר הבאגים גדול מאפס?"
פייתון יבדוק את המצב בזמן אמת, ויחזיר כפלט, ערך בוליאני:
True (אמת / כן) – התנאי התקיים (למשל, הכפתור שם, הטסט עבר).
False (שקר / לא) – התנאי לא התקיים (למשל, הכפתור חסר, הטסט נכשל).
במילים אחרות, Boolean הוא הבסיס לכל קבלת ההחלטות בקוד, והוא זה שקובע בסוף אם הטסט יהיה ירוק (**Pass**) או אדום (**Fail**).

כתבו את שורת הקוד הזאת כדי להחזיר **True**: `print (5>3)`

כתבו שורת קוד כדי להחזיר **False**: `print (5<3)`